Lambda-Ausdrücke und funktionale Interfaces

TL;DR
Mit einer anonymen inneren Klasse erstellt man gewissermaßen ein Objekt einer “Wegwerf”-Klasse: Man leitet *on-the-fly* von einem Interface ab oder erweitert eine Klasse und implementiert die benötigten Methoden und erzeugt von dieser Klasse sofort eine Instanz (Objekt). Diese neue Klasse ist im restlichen Code nicht sichtbar.
Anonyme innere Klassen sind beispielsweise in Swing recht nützlich, wenn man einer Komponente einen Listener mitgeben will: Hier erzeugt man eine anonyme innere Klasse basierend auf dem passenden Listener-Interface, implementiert die entsprechenden Methoden und übergibt das mit dieser Klasse erzeugte Objekt als neuen Listener der Swing-Komponente.
Mit Java 8 können unter gewissen Bedingungen diese anonymen inneren Klassen zu Lambda-Ausdrücken (und Methoden-Referenzen) vereinfacht werden. Dazu muss die anonyme innere Klasse ein sogenanntes **funktionales Interface** implementieren.
Funktionale Interfaces sind Interfaces mit *genau einer abstrakten Methode*. Es können beliebig viele Default-Methoden im Interface enthalten sein, und es können `public` sichtbare abstrakte Methoden von `java.lang.Object` geerbt/überschrieben werden.
Die Lambda-Ausdrücke entsprechen einer anonymen Methode: Die Parameter werden aufgelistet (in Klammern), und hinter einem Pfeil kommt entweder *ein* Ausdruck (Wert - gleichzeitig Rückgabewert des Lambda-Ausdrucks) oder beliebig viele Anweisungen (in geschweiften Klammern, mit Semikolon):

- Form 1: `(parameters)-> expression`
- Form 2: `(parameters)-> { statements; }`

Der Lambda-Ausdruck muss von der Signatur her genau der einen abstrakten Methode im unterliegenden funktionalen Interface entsprechen.

Videos

- VL Lambda-Ausdrücke und funktionale Interfaces
- Demo Anonyme innere Klasse
- Demo Lambda-Ausdruck
- Demo Funktionale Interfaces selbst definiert
- Demo Vordefinierte funktionale Interfaces im JDK

Unterlagen

- PDF-Version

Problem: Sortieren einer Studi-Liste

```
List<Studi> sl = new ArrayList<>();

// Liste sortieren?
sl.sort(???);  // Parameter: java.util.Comparator<Studi>
```

```
public class MyCompare implements Comparator<Studi> {
    @Override  public int compare(Studi o1, Studi o2) {
```

```
        return o1.getCredits() - o2.getCredits();
    }
}
```

```
// Liste sortieren?
MyCompare mc = new MyCompare();
sl.sort(mc);
```

Da `Comparator<T>` ein Interface ist, muss man eine extra Klasse anlegen, die die abstrakte Methode aus dem Interface implementiert und ein Objekt von dieser Klasse erzeugen und dieses dann der `sort()`-Methode übergeben.

Die Klasse bekommt wie in Java üblich eine eigene Datei und ist damit in der Package-Struktur offen sichtbar und "verstopft" mir damit die Strukturen: Diese Klasse ist doch nur eine Hilfsklasse … Noch schlimmer: Ich brauche einen Namen für diese Klasse!

Den ersten Punkt könnte man über verschachtelte Klassen lösen: Die Hilfsklasse wird innerhalb der Klasse definiert, die das Objekt benötigt. Für den zweiten Punkt brauchen wir mehr Anlauf …

Erinnerung: Verschachtelte Klassen ("*Nested Classes*")

Man kann Klassen innerhalb von Klassen definieren: Verschachtelte Klassen.

- Implizite Referenz auf Instanz der äußeren Klasse, Zugriff auf **alle** Elemente
- **Begriffe**:
  - "normale" innere Klassen: "*inner classes*"
  - statische innere Klassen: "*static nested classes*"
- Einsatzzweck:
  - Hilfsklassen: Zusätzliche Funktionalität kapseln; Nutzung **nur** in äußerer Klasse
  - Kapselung von Rückgabewerten

Sichtbarkeit: Wird u.U. von äußerer Klasse "überstimmt"

Innere Klassen ("*Inner Classes*")

- Objekt der äußeren Klasse muss existieren
- Innere Klasse ist normales Member der äußeren Klasse
- Implizite Referenz auf Instanz äußerer Klasse
- Zugriff auf **alle** Elemente der äußeren Klasse
- Sonderfall: Definition innerhalb von Methoden ("local classes")
  - Nur innerhalb der Methode sichtbar
  - Kennt zusätzlich `final` Attribute der Methode

Beispiel:

```java
public class Outer {
    ...
    private class Inner {
        ...
    }

    Outer.Inner inner = new Outer().new Inner();
}
```

Beispiel mit Iterator als innere Klasse: nested.StudiListNested (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/nested/StudiListNested.java)

Statische innere Klassen ("*Static Nested Classes*")

- Keine implizite Referenz auf Objekt
- Nur Zugriff auf Klassenmethoden und -attribute

Beispiel:

```java
class Outer {
    ...
    static class StaticNested {
        ...
    }
}

Outer.StaticNested nested = new Outer.StaticNested();
```

Lösung: Comparator als anonyme innere Klasse

```java
List<Studi> sl = new ArrayList<>();

// Parametrisierung mit anonymer Klasse
sl.sort(
        new Comparator<Studi>() {
            @Override
            public int compare(Studi o1, Studi o2) {
                return o1.getCredits() - o2.getCredits();
            }
        });  // Semikolon nicht vergessen!!!
```

=> Instanz einer anonymen inneren Klasse, die das Interface `Comparator<Studi>` implementiert

- Für spezielle, einmalige Aufgabe: nur eine Instanz möglich
- Kein Name, kein Konstruktor, oft nur eine Methode
- Müssen Interface implementieren oder andere Klasse erweitern
  - Achtung Schreibweise: ohne **`implements`** oder **`extends`**!
- Konstruktor kann auch Parameter aufweisen

- Zugriff auf alle Attribute der äußeren Klasse plus alle `final` lokalen Variablen
- Nutzung typischerweise bei GUIs: Event-Handler etc.

Demo: nested.DemoAnonymousInnerClass (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/nested/DemoAnonymousInnerClass.java)

Vereinfachung mit Lambda-Ausdruck

```
List<Studi> sl = new ArrayList<>();

// Parametrisierung mit anonymer Klasse
sl.sort(
        new Comparator<Studi>() {
            @Override
            public int compare(Studi o1, Studi o2) {
                return o1.getCredits() - o2.getCredits();
            }
        });  // Semikolon nicht vergessen!!!


// Parametrisierung mit Lambda-Ausdruck
sl.sort( (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits() );
```

Anmerkung: Damit für den Parameter alternativ auch ein Lambda-Ausdruck verwendet werden kann, muss der erwartete Parameter vom Typ her ein "**funktionales Interface**" (s.u.) sein!

Demo: nested.DemoLambda (https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/java-modern/src/nested/DemoLambda.java)

Syntax für Lambdas

```
(Studi o1, Studi o2)  ->  o1.getCredits() - o2.getCredits()
```

Ein Lambda-Ausdruck ist eine Funktion ohne Namen und besteht aus drei Teilen:

1. Parameterliste (in runden Klammern),
2. Pfeil
3. Funktionskörper (rechte Seite)

Falls es *genau einen* Parameter gibt, *können* die runden Klammern um den Parameter entfallen.

Dabei kann der Funktionskörper aus *einem Ausdruck* ("*expression*") bestehen oder einer *Menge von Anweisungen* ("*statements*"), die dann in geschweifte Klammern eingeschlossen werden müssen (Block mit Anweisungen).

Der Wert des Ausdrucks ist zugleich der Rückgabewert des Lambda-Ausdrucks.

Varianten:

- `(parameters)-> expression`

- `(parameters)-> { statements; }`

Quiz: Welches sind keine gültigen Lambda-Ausdrücke?

1. `() -> {}`
2. `() -> "wuppie"`
3. `() -> { return "fluppie"; }`
4. `(Integer i)-> return i + 42;`
5. `(String s)-> { "foo"; }`
6. `(String s)-> s.length()`
7. `(Studi s)-> s.getCredits()> 300`
8. `(List<Studi> sl)-> sl.isEmpty()`
9. `(int x, int y)-> { System.out.println("Erg: "); System.out.println(x+y); }`
10. `() -> new Studi()`
11. `s -> s.getCps()> 100 && s.getCps()< 300`
12. `s -> { return s.getCps()> 100 && s.getCps()< 300; }`

Auflösung:

(4) und (5): `return` ist eine Anweisung, d.h. bei (4) fehlen die geschweiften Klammern. `"foo"` ist ein String und als solcher ein Ausdruck, d.h. hier sind die geschweiften Klammern zu viel (oder man ergänze den String mit einem `return`, also `return "foo";` …).

Definition "Funktionales Interface" ("*functional interfaces*")

```
@FunctionalInterface
public interface Wuppie<T> {
    int wuppie(T obj);
    boolean equals(Object obj);
    default int fluppie() { return 42; }
}
```

`Wuppie<T>` ist ein **funktionales Interface** ("*functional interface*") (seit Java 8)

- Hat **genau *eine* abstrakte Methode**
- Hat evtl. weitere Default-Methoden
- Hat evtl. weitere abstrakte Methoden, die `public` Methoden von `java.lang.Object` überschreiben

Die Annotation `@FunctionalInterface` selbst ist nur für den Compiler: Falls das Interface *kein* funktionales Interface ist, würde er beim Vorhandensein dieser Annotation einen Fehler werfen. Oder anders herum: Allein durch das Annotieren mit `@FunctionalInterface` wird aus einem Interface noch kein funktionales Interface! Vergleichbar mit `@Override` …

Während man für eine anonyme Klasse lediglich ein "normales" Interface (oder eine Klasse) benötigt, braucht man für Lambda-Ausdrücke zwingend ein passendes funktionales Interface!

Anmerkung: Es scheint keine einheitliche deutsche Übersetzung für den Begriff *functional interface* zu geben. Es wird häufig mit “funktionales Interface”, manchmal aber auch mit “Funktionsinterface” übersetzt.

Das in den obigen Beispielen eingesetzte Interface `java.util.Comparator<T>` ist also ein funktionales Interface: Es hat nur *eine* eigene abstrakte Methode `int compare(T o1, T o2);`.

Im Package java.util.function sind einige wichtige funktionale Interfaces bereits vordefiniert, beispielsweise `Predicate` (Test, ob eine Bedingung erfüllt ist) und `Function` (verarbeite einen Wert und liefere einen passenden Ergebniswert). Diese kann man auch in eigenen Projekten nutzen!

Quiz: Welches ist kein funktionales Interface?

```
public interface Wuppie {
    int wuppie(int a);
}

public interface Fluppie extends Wuppie {
    int wuppie(double a);
}

public interface Foo {
}

public interface Bar extends Wuppie {
    default int bar() { return 42; }
}
```

Auflösung:

- `Wuppie` hat *genau eine* abstrakte Methode => funktionales Interface
- `Fluppie` hat zwei abstrakte Methoden => **kein** funktionales Interface
- `Foo` hat gar keine abstrakte Methode => **kein** funktionales Interface
- `Bar` hat *genau eine* abstrakte Methode (und eine Default-Methode) => funktionales Interface

Lambdas und funktionale Interfaces: Typprüfung

```
interface java.util.Comparator<T> {
    int compare(T o1, T o2);    // abstrakte Methode
}
```

```
// Verwendung ohne weitere Typinferenz
Comparator<Studi> c1 = (Studi o1, Studi o2) -> o1.getCredits() - o2.getCredits
    ();

// Verwendung mit Typinferenz
Comparator<Studi> c2 = (o1, o2) -> o1.getCredits() - o2.getCredits();
```

Der Compiler prüft in etwa folgende Schritte, wenn er über einen Lambda-Ausdruck stolpert:

1. In welchem Kontext habe ich den Lambda-Ausdruck gesehen?
2. OK, der Zieltyp ist hier `Comparator<Studi>`.
3. Wie lautet die **eine** abstrakte Methode im `Comparator<T>`-Interface?
4. OK, das ist `int compare(T o1, T o2);`
5. Da `T` hier an `Studi` gebunden ist, muss der Lambda-Ausdruck der Methode `int compare(Studi o1, Studi o2);` entsprechen: 2x `Studi` als Parameter und als Ergebnis ein `int`
6. Ergebnis:
   a) Cool, passt zum Lambda-Ausdruck `c1`. Fertig.
   b) D.h. in `c2` müssen `o1` und `o2` vom Typ `Studi` sein. Cool, passt zum Lambda-Ausdruck `c2`. Fertig.

Wrap-Up

- Anonyme Klassen: "Wegwerf"-Innere Klassen
  - Müssen Interface implementieren oder Klasse erweitern
- Java8: **Lambda-Ausdrücke** statt anonymer Klassen (**funktionales Interface nötig**)
  - Zwei mögliche Formen:
    * Form 1: `(parameters)-> expression`
    * Form 2: `(parameters)-> { statements; }`
  - Im jeweiligen Kontext muss ein **funktionales Interface** verwendet werden, d.h. ein Interface mit **genau** einer abstrakten Methode
  - Der Lambda-Ausdruck muss von der Signatur her dieser einen abstrakten Methode entsprechen

Zum Nachlesen

- Oracle Corporation (2024)
- Urma, Fusco, und Mycroft (2014, Kap. 3)
- Ullenboom (2021, Kap. 12)

Lernziele

- k2: Funktionales Interfaces (Definition)
- k3: Einsatz innerer und anonymer Klassen
- k3: Erstellen eigener funktionaler Interfaces
- k3: Einsatz von Lambda-Ausdrücken

Challenges

Beispiel aus einem Code-Review im Dungeon-CampusMinden/Dungeon

Erklären Sie folgenden Code:

```
public interface IFightAI {
    void fight(Entity entity);
}

public class AIComponent extends Component {
    private final IFightAI fightAI;

    fightAI =
                entity1 -> {
                    System.out.println("TIME TO FIGHT!");
                    // todo replace with melee skill
                };
}
```

Sortieren mit Lambdas und funktionalen Interfaces

Betrachten Sie die Klasse Student.

1. Definieren Sie eine Methode, die das Sortieren einer `Student`-Liste erlaubt. Übergeben Sie die Liste als Parameter.
2. Schaffen Sie es, das Sortierkriterium ebenfalls als Parameter zu übergeben (als Lambda-Ausdruck)?
3. Definieren Sie eine weitere Methode, die wieder eine `Student`-Liste als Parameter bekommt und liefern sie das erste `Student`-Objekt zurück, welches einer als Lambda-Ausdruck übergebenen Bedingung genügt.
4. Definieren Sie noch eine Methode, die wieder eine `Student`-Liste als Parameter bekommt sowie einen Lambda-Ausdruck, welcher aus einem `Student`-Objekt ein Objekt eines anderen Typen `T` berechnet. Wenden Sie in der Methode den Lambda-Ausdruck auf jedes Objekt der Liste an und geben sie die resultierende neue Liste als Ergebnis zurück.

Verwenden Sie in dieser Aufgabe jeweils Lambda-Ausdrücke. Rufen Sie alle drei/vier Methoden an einem kleinen Beispiel auf.

Quellen

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Ullenboom, C. 2021. *Java ist auch eine Insel*. 16. Aufl. Rheinwerk-Verlag. https://openbook.rheinwerk-verlag.de/javainsel/index.html.

Urma, R.-G., M. Fusco, und A. Mycroft. 2014. *Java 8 in Action: Lambdas, Streams, and Functional-Style Programming*. Manning Publications.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** ba2c853 (Lecture: remove 'quizzes' from the metadata to avoid any confusion with the new lecture quizzes, 2025-04-09)